

Group 2 — SSO Integration Guide

Government Asset Management System

Quick Summary — What We Each Do

Who	Responsibility
Group 1 (us)	We manage users, roles, ministries, and authentication. We run Keycloak SSO + the verify-token API.
Group 2 (you)	You build your asset register UI. Your users log in through our Keycloak page. You call our API to find out who the user is.

You do NOT build: login page, password storage, user management, role management, ministry management.

We handle all of that. We tell you: "This user is Amina Hassan, she is MINISTRY_ADMIN for Ministry of Health." You just check the role and show/hide buttons accordingly.

Table of Contents

- [1. What We Give You](#)
 - [2. What You Give Us](#)
 - [3. Prerequisites — Both Sides Must Be Accessible](#)
 - [4. How Login Works \(Step by Step\)](#)
 - [5. Callback Handler — What Your Backend Does](#)
 - [6. Verify Token — Getting the User's Role and Ministry](#)
 - [7. The 5 Roles and What Your UI Should Do](#)
 - [8. Test Accounts You Can Use](#)
 - [9. Logout](#)
 - [10. Common Problems and Fixes](#)
 - [11. Integration Checklist](#)
-

1. What We Give You

After you send us your details (see Section 2), we set up your application in our Keycloak and give you:

What we give you	Example	Why you need it
Client ID	<code>group2-app</code>	Identifies your app to our Keycloak
Client Secret	<code>a1b2c3d4-...</code>	Proves your app is authorized (keep this secret on your backend)

What we give you	Example	Why you need it
Our Keycloak Server URL	<code>https://govasset-keycloak.up.railway.app</code>	Base URL for all authentication requests
Our Django API URL	<code>https://govasset-api.up.railway.app</code>	Base URL for verify-token API calls

From these two base URLs, you can construct all the endpoints you need:

Endpoint	How to build it	Purpose
SSO Login Page	<code>{Keycloak URL}/realms/govasset/protocol/openid-connect/auth</code>	Send users here to log in
Token Exchange	<code>{Keycloak URL}/realms/govasset/protocol/openid-connect/token</code>	Exchange auth code for tokens
Verify Token	<code>{Django URL}/api/auth/verify-token/</code>	Get user's name, role, and ministry
Logout	<code>{Keycloak URL}/realms/govasset/protocol/openid-connect/logout</code>	Log user out of all systems

2. What You Give Us

Before we can set anything up, we need these **3 things** from you:

What we need	Example	Why
Your app name	<code>Group 2 Asset Register</code>	We label your client in our admin panel
Your callback URL	<code>https://your-system.com/auth/callback</code> or <code>https://your-ngrok.ngrok-free.app/auth/callback</code>	Where we send users after they log in on our Keycloak page. Must be a real public URL — not <code>localhost</code> .
Your logout URL	<code>https://your-system.com</code>	Where we send users after they log out

Important: Your callback URL must be a real, accessible URL — not `localhost`. See Section 3.

3. Prerequisites — Both Sides Must Be Accessible

This integration is **two-way**. Your system and our system both need to talk to each other over the internet.

Our side (we handle this)

Both services are deployed on **Railway** (cloud hosting) and have permanent public URLs that never change:

Our service	URL
Keycloak SSO	https://govasset-keycloak.up.railway.app
Django API	https://govasset-api.up.railway.app

Both services run 24/7 — no laptop needed, no URL changes.

Your side (you must do this)

Your system must be accessible from the internet so that **Keycloak can redirect users back to your callback URL** after they log in.

Your system	How to expose it	What we need
Your callback URL	Host on a public server, OR use ngrok during development	The full callback URL (e.g. https://your-system.com/auth/callback)

How to use ngrok on your side (for development testing):

1. Download ngrok from <https://ngrok.com>
2. Run your system on a port (e.g., port 3000)
3. In a terminal: `ngrok http 3000`
4. ngrok gives you a URL: <https://xxxxxxx.ngrok-free.app>
5. Send us that URL as your callback URL (append `/auth/callback` to it)

Why this is needed:

```
User opens your app → you redirect to our Keycloak → user logs in
→ Keycloak redirects back to YOUR callback URL
→ If your callback URL is localhost, Keycloak cannot reach it
→ ngrok makes your localhost reachable from the internet
```

4. How Login Works (Step by Step)

Here is exactly what happens when a user clicks "Login" on your website:

```
Step 1: User clicks "Login with Government SSO" on your site
      ↓
Step 2: Your app redirects the user's browser to our Keycloak login page
      ↓
Step 3: User sees our government-branded login page, types username + password
      ↓
Step 4: Keycloak verifies the credentials
      ↓
Step 5: Keycloak redirects the browser back to YOUR callback URL
        with an authorization code: https://your-system.com/auth/callback?
```

```
code=abc123
```

↓

Step 6: YOUR BACKEND (server-side, NOT browser) calls our token endpoint to exchange the code for tokens

↓

Step 7: Keycloak returns: access_token + refresh_token + id_token

↓

Step 8: YOUR BACKEND calls our verify-token API with the access_token to get the user's name, role, and ministry

↓

Step 9: You create a session and show your dashboard to the user

How to redirect the user to Keycloak (Step 2):

Put a button on your page that links to:

```
https://{OUR_KEYCLOAK_URL}/realms/govasset/protocol/openid-connect/auth?
  client_id=group2-app&
  redirect_uri=https://{YOUR_CALLBACK_URL}&
  response_type=code&
  scope=openid+profile+email&
  state=YOUR_RANDOM_STRING
```

Parameters explained:

Parameter	Value	Why
<code>client_id</code>	<code>group2-app</code>	Identifies your app (we give you this)
<code>redirect_uri</code>	Your callback URL	Where to send the user back (you give us this)
<code>response_type</code>	<code>code</code>	Use the authorization code flow
<code>scope</code>	<code>openid profile email</code>	Get user's basic info
<code>state</code>	Random string you generate	Security — prevents CSRF attacks

Important: Generate a random `state` string for each login request. Store it in the user's session. When Keycloak sends the user back, verify the `state` matches.

5. Callback Handler — What Your Backend Does

When Keycloak redirects the user back to your callback URL, your backend must handle it. Here is exactly what your code should do:

Step A — Verify the `state` parameter

```
Check: Does the "state" value in the URL match what you stored in Step 2?
YES → Continue
```

```
NO → Reject the request (possible CSRF attack)
```

Step B — Exchange the code for tokens

Your backend makes a **server-to-server** POST request (not from the browser):

```
POST https://{OUR_KEYCLOAK_URL}/realms/govasset/protocol/openid-connect/token
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&client_id=group2-app
&client_secret=YOUR_CLIENT_SECRET
&code=THE_CODE_FROM_THE_URL
&redirect_uri=https://{YOUR_CALLBACK_URL}
```

What you get back:

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIs... ",
  "expires_in": 300,
  "refresh_token": "eyJhbGciOiJIUzUxMiIs... ",
  "id_token": "eyJhbGciOiJSUzI1NiIs... "
}
```

Save the **access_token** — you need it for the next step.

Save the **refresh_token** — you use it when the access token expires (5 minutes).

Step C — Call our API to get the user's identity

```
GET https://{OUR_DJANGO_URL}/api/auth/verify-token/
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
```

What you get back:

```
{
  "valid": true,
  "user": {
    "id": 29,
    "username": "moh_admin",
    "full_name": "Amina Hassan",
    "email": "amina@moh.go.tz",
    "role": "MINISTRY_ADMIN",
    "ministry_schema": "moh_schema",
    "ministry": "Ministry of Health"
  }
}
```

```
}  
}
```

This tells you everything you need:

- **full_name** — Show on your dashboard header: "Welcome, Amina Hassan"
- **role** — Controls permissions (see Section 7)
- **ministry** — Which ministry's data to show
- **ministry_schema** — Internal identifier for scoping queries

Step D — Create a session and show your dashboard

Store the user info in your session. The user is now logged in.

6. Verify Token — Getting the User's Role and Ministry

This is the most important API call. You will call it:

1. **After login** (to get the initial user info)
2. **On every page load** (to verify the user still has permission)
3. **Before any sensitive action** (like delete, approve, or financial transactions)

Request

```
GET https://{OUR_DJANGO_URL}/api/auth/verify-token/  
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Success Response

```
{  
  "valid": true,  
  "user": {  
    "id": 29,  
    "username": "moh_admin",  
    "full_name": "Amina Hassan",  
    "email": "amina@moh.go.tz",  
    "role": "MINISTRY_ADMIN",  
    "ministry_schema": "moh_schema",  
    "ministry": "Ministry of Health"  
  }  
}
```

Error Response (401)

```
{
  "detail": "Given token not valid for any token type"
}
```

If you get 401, the token has expired. Use the refresh token to get a new one:

```
POST https://{OUR_KEYCLOAK_URL}/realms/govasset/protocol/openid-connect/token
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token
&client_id=group2-app
&client_secret=YOUR_CLIENT_SECRET
&refresh_token=THE_REFRESH_TOKEN
```

If the refresh also fails, redirect the user back to the Keycloak login page.

7. The 5 Roles and What Your UI Should Do

Our system has exactly **5 roles**. Every user in the entire project has one of these.

Role	Who has it	What your UI should allow
SUPER_ADMIN	Platform administrators	Full access. Can see all ministries' data. Your system may show a ministry selector.
MINISTRY_ADMIN	Ministry IT staff	Full access within their own ministry. Can create, edit, delete assets. Can manage users.
AGENCY_MANAGER	Agency heads (hospital directors, etc.)	Can create and edit assets. Can manage facility clerks. Cannot delete.
FACILITY_CLERK	Frontline staff (store clerks, etc.)	Can register and edit assets they created. Cannot delete or manage users.
AUDITOR	Government auditors	Read-only. Can view everything but cannot create, edit, or delete.

Rule: Check the **role** field on every request, not just at login. If the user's role changes (admin promotes them), the next verify-token call will reflect the change.

8. Test Accounts You Can Use

All passwords: **Admin@123**

Username	Full Name	Role	Ministry	What to test
superadmin	System Administrator	SUPER_ADMIN	All ministries	Should see all data
moh_admin	Amina Hassan	MINISTRY_ADMIN	Ministry of Health	Should see MOH data only, full access
mnh_manager	John Mwangi	AGENCY_MANAGER	Ministry of Health	Should see MOH data, limited management
rad_clerk	Asha Salum	FACILITY_CLERK	Ministry of Health	Should see MOH data, no delete
moh_auditor	David Mushi	AUDITOR	Ministry of Health	Should see MOH data, read-only
mof_admin	Grace Mbwilo	MINISTRY_ADMIN	Ministry of Finance	Should see MOF data only

Test sequence to verify your integration:

1. Log in as moh_admin
 - Your dashboard should show: "Amina Hassan" + "MINISTRY_ADMIN" + "Ministry of Health"
 - You should see Ministry of Health data (not Finance data)
 - You should have full CRUD access
2. Log in as rad_clerk
 - Your dashboard should show: "Asha Salum" + "FACILITY_CLERK" + "Ministry of Health"
 - Delete button should be hidden or disabled
3. Log in as moh_auditor
 - Your dashboard should show: "David Mushi" + "AUDITOR" + "Ministry of Health"
 - All create/edit/delete buttons should be hidden
4. Log in as mof_admin
 - Your dashboard should show: "Grace Mbwilo" + "MINISTRY_ADMIN" + "Ministry of Finance"
 - You should see Ministry of Finance data (not Health data)

9. Logout

When the user clicks "Logout" on your site, redirect their browser to:


```
https://{OUR_KEYCLOAK_URL}/realms/govasset/protocol/openid-connect/logout?
id_token_hint=THE_ID_TOKEN_FROM_LOGIN&
post_logout_redirect_uri=https://{YOUR_LOGOUT_URL}
```

This logs the user out of **our Keycloak session**. Since all groups use the same Keycloak, this logs them out of every group's system at once.

10. Common Problems and Fixes

Problem	Cause	Fix
User sees "Invalid redirect URI" after login	The callback URL you gave us doesn't match the URL Keycloak is redirecting to	Check both are identical (including trailing slashes, http vs https)
Your callback returns "state mismatch"	You didn't generate/stored the state parameter	Generate a random state for each login, store it in session, verify on return
verify-token returns 401	Access token expired	Use refresh token to get a new access token
refresh also returns error	Refresh token also expired	Redirect user back to Keycloak login page
Keycloak page doesn't load	Our server is down	Contact us — our services run 24/7 on Railway
User gets redirected back to your site but your site shows an error	Your callback handler has a bug	Check your server logs. The code exchange or verify-token call may be failing
You get "User not found" from verify-token	User exists in Keycloak but not in our Django database	Contact us — we need to create the user in our system

11. Integration Checklist

Before you start — send us:

- ☐ Your app name (e.g., "Group 2 Asset Register")
- ☐ Your callback URL (must be publicly accessible, NOT localhost)
- ☐ Your logout redirect URL

We send back to you:

- ☐ Client ID
- ☐ Client Secret
- ☐ Our Keycloak URL: <https://govasset-keycloak.up.railway.app> (permanent)

- ☐ Our Django API URL: <https://govasset-api.up.railway.app> (permanent)

You implement on your end:

- ☐ Host your system or use ngrok so your callback URL is publicly accessible
- ☐ Add "Login with Government SSO" button on your site
- ☐ Generate random `state` parameter for each login request
- ☐ Create a `/auth/callback` endpoint in your backend
- ☐ Code exchange: POST to our token endpoint with `grant_type=authorization_code`
- ☐ Call `GET /api/auth/verify-token/` with the access token
- ☐ Use `user.role` to control permissions (hide/show buttons)
- ☐ Use `user.ministry` to filter data (never show cross-ministry data)
- ☐ Implement token refresh (when access token expires)
- ☐ Handle logout redirect to our Keycloak logout endpoint

Summary — What You Need to Know

Topic	Answer
Do you need to build a login page?	No. Your users log in on our Keycloak page.
Do you need to store passwords?	No. We handle all password management.
Do you need to manage users?	No. We create and manage all users.
Do you need to manage roles?	No. We assign roles. You just read the <code>role</code> field.
Do you need to manage ministries?	No. We manage ministries. You just read the <code>ministry_schema</code> field.
Do you need to host your system?	Yes. Use a public server or ngrok so Keycloak can redirect users back to your callback URL.
What do you actually build?	Your asset register UI + a callback handler + permission checks based on <code>role</code> .